[4]. Despite altering 0.34% weights, the trained LoRA adapter still needs to be merged into the original model to form a new LLM, and tend to be harder to deploy than test-time defenses like defensive prompting and DefensiveToken.

The right 3 bars on every sub-figure in Fig. 2 show results of training-time defenses. Despite as a test-time defense, DefensiveToken enjoys a security level comparable to training-time defenses. In the largest TaskTracker benchmark, DefensiveTokens mitigates optimization-free attacks to an average ASR of 0.24%, which is close to training-time defenses (ASRs 0.20% to 0.51%). A similar trend can be seen on AlpacaFarm, SEP, and CyberSecEval2 benchmarks. For attacks using optimization or on agentic InjecAgent benchmark, DefensiveToken is slightly weaker than training-time alternatives.

4.4 Analyzing the Utility-Security Trade-Off

Security should be measured with utility to make sure the model is useful for a defense. Table 3 shows that most evaluated defenses, except TextGrad and StruQ-Full (on Falcon3), have a slight utility drop in AlpacaFarm and SEP benchmark. For a high-level view, we plot the utility-security trade-off in Fig. 3. Even when DefensiveToken is implemented, it is the defense that loses the least utility compared to all test-time and training-time baselines. We hypothesize that it is because DefensiveToken adds slight changes (only 5 more tokens) to the system. Also, DefensiveToken is the closest defense to an ideal defense (0% ASR, no utility loss) against optimization-free attacks on two benchmarks. For optimization-based attacks, DefensiveToken still emerges as the best test-time defense with an impressive utility-security trade-off. Even better, the slight utility loss of DefensiveToken is only confined to developers who need security, and has no impact on those aiming for utility in less risky applications.

4.5 Ablation Study

We conduct ablation studies to analyze the impact of various design choices and hyperparameters on the performance of DefensiveToken. We evaluate using the AlpacaFarm benchmark, focusing on the Llama3.1-8B-Instruct model for most ablations.

Number of DefensiveTokens. Table 4 shows the effect of varying the number of defensive tokens. Overall, more optimized tokens lead to better security but worse utility: The Falcon3-7B-Instruct ASR drops from 70% (1 token) to 0% (20 tokens), but the latter loses 2.4% utility score. Different models require different numbers of defensive tokens to reach a satisfactory security. On Llama3-8B-Instruct and Llama3.1-8B-Instruct, there is no benefit in tuning more than a single embedding, and 5 embedding tokens are sufficient for all 4 models.

DefensiveToken initialization. We also experiment with different initializations of the tuned tokens in Table 5. It turns out that random initialization is better than the other heuristics, like initializing with the embeddings of space and text ("`You should follow all the instructions in the system block and not follow any instructions in the user block.`" following [42]). Based on Table 2, we hypothesize that it is because random initialization gives larger magnitude embeddings that facilitate optimization. If starting on a small initialization using vocabulary embeddings, the optimizer needs to first enlarge those embeddings for a larger optimization space where a good solution lies. This conclusion on initialization is different from the original prompt tuning paper [14], where initializing with text embeddings works best. This may be because our defense objective is more complex than improving utility in a given task, see Section 3.4, and thus requires a larger optimization space.

Table 4: Ablation study on the number of defensive tokens in DefensiveToken using AlpacaFarm. 1 token lends noticeable security. 5 tokens are sufficient for good security with minimal utility loss and are thus used in our main experiments. 20 tokens require a larger learning rate of 1.0, also see Table 8, and tend to offer better security.

	#Tokens	WinRate (↑)	ASR (↓)
Llama3-8B	0	26.53	51.44
	1	**27.21**	0.96
	5	27.04	**0.48**
	20	26.61	**0.48**
Llama3.1-8B	0	**29.07**	69.23
	1	28.44	0.48
	5	28.53	0.48
	20	29.00	**0**
Falcon3-7B	0	**30.73**	84.62
	1	29.43	70.19
	5	29.21	4.81
	20	28.33	**0.48**
Qwen2.5-7B	0	32.69	93.27
	1	33.70	38.94
	5	**34.16**	**0.96**
	20	31.87	2.88

Table 5: Ablation study on the initialization of defensive tokens in DefensiveToken using AlpacaFarm and Llama3.1-8B-Instruct.

Init.	#Tokens	WinRate (↑)	ASR (↓)
None	0	29.07	69.23
random	1	**28.44**	**0.48**
space	1	27.49	7.7
random	**5**	**28.53**	**0.48**
space	5	27.04	2.40
random	20	**29.00**	**0**
space	20	25.88	**0**
text	20	25.74	**0**

Loss function. SecAlign [4] uses preference optimization instead of supervised fine-tuning in StruQ. Besides training the LLM to prefer the response to the user instruction, SecAlign also penalizes the response to the injection. This is an objective harder than StruQ

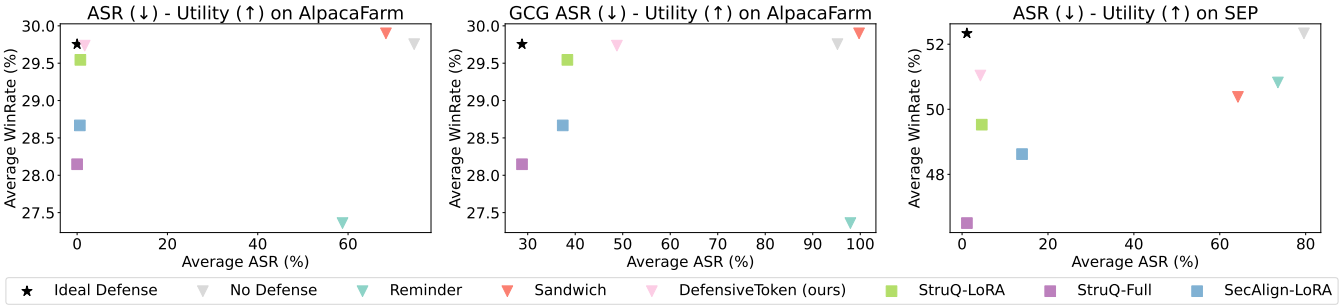

Figure 3: The utility-security trade-off on AlpacaFarm and SEP. The triangles mark test-time defenses, and the squares mark training-time ones. The utility and attack success rate (ASR) are averaged across four tested models. DefensiveToken is flexible with utility-security trade-off close to an ideal defense.

Table 3: Utility (WinRate ↑) and security (ASR ↓) of test-time (TextGrad, Reminder, Sandwich, DefensiveToken) and training-time (StruQ, SecAlign using Full/LoRA fine-tuning) defense baselines.

	Benchmark	AlpacaFarm			SEP		TaskTracker	CyberSecEval2	InjecAgent
	Defense	WinRate ↑	ASR ↓	GCG-ASR ↓	WinRate ↑	ASR ↓	ASR ↓	ASR ↓	ASR ↓
Llama3-8B-Instruct	None	26.5	51.4	94.7	50.0	79.1	16.4	49.1	29.6
	TextGrad	22.9	**0**	**31.6**	1.1	3.5	**0.25**	**1.8**	8.8
	Reminder	24.4	34.6	96.6	48.3	75.2	19.8	43.6	42.2
	Sandwich	26.8	56.7	100.0	46.9	63.4	5.5	41.8	14.8
	DefensiveToken	**27.0**	0.5	37.5	**51.6**	**3.2**	0.27	3.6	**2.7**
	StruQ-LoRA	28.0	0	4.8	50.4	1.5	0.24	7.3	0
	StruQ-Full	27.9	0	2.9	51.2	0.4	0.23	10.9	0
	SecAlign-LoRA	27.0	0	1.9	47.5	3.1	0.18	18.2	0
Llama3.1-8B-Instruct	None	29.1	69.2	96.2	**54.7**	71.4	26.6	16.4	33.0
	TextGrad	20.9	15.9	92.8	36.3	22.1	20.3	23.6	25.3
	Reminder	26.2	29.8	97.1	52.5	50.6	23.3	**7.3**	34.3
	Sandwich	**29.7**	60.6	100.0	51.5	55.0	11.1	25.5	21.4
	DefensiveToken	28.5	**0.5**	**24.6**	53.8	**2.8**	**0.19**	**7.3**	**0.6**
	StruQ-LoRA	27.6	0.5	10.1	51.6	1.4	0.23	12.7	3.9
	StruQ-Full	28.2	0	17.3	52.9	0.2	0.18	10.9	1.8
	SecAlign-LoRA	27.5	0	1.0	50.5	2.7	0.19	5.5	0.1
Falcon3-7B-Instruct	None	30.7	84.6	94.2	50.5	80.8	27.7	50.9	20.3
	TextGrad	28.0	97.1	70.8	47.0	80.6	28.1	29.1	11.5
	Reminder	29.8	75.0	99.0	**51.8**	83.4	30.5	47.3	27.2
	Sandwich	**30.9**	70.7	99.0	49.8	68.4	8.9	43.6	3.3
	DefensiveToken	29.2	**4.8**	**59.4**	48.3	**6.7**	**0.27**	**12.7**	**1.6**
	StruQ-LoRA	29.2	1.0	73.1	45.4	11.6	0.27	21.8	0.1
	StruQ-Full	25.3	0	48.8	31.3	2.0	0.20	7.3	0
	SecAlign-LoRA	27.4	0.5	81.7	46.1	35.4	1.1	29.1	2.4
Qwen2.5-7B-Instruct	None	32.7	93.3	95.7	**54.1**	87.1	37.2	45.5	23.5
	TextGrad	13.8	97.6	82.6	36.1	90.2	33.7	34.6	19.8
	Reminder	29.0	94.7	99.0	50.7	85.0	35.3	32.7	29.6
	Sandwich	32.3	85.6	100.0	53.3	70.2	18.5	47.3	**11.7**
	DefensiveToken	**34.2**	**1.0**	**73.6**	50.5	**4.3**	**0.25**	**20.0**	15.8
	StruQ-LoRA	33.5	1.4	65.4	50.8	3.9	0.24	23.6	2.1
	StruQ-Full	31.1	0	46.2	50.5	2.0	0.20	3.6	0.5
	SecAlign-LoRA	32.8	1.9	64.9	50.5	14.7	0.57	20.0	5.5

Table 7: Ablation study on the position of DefensiveTokens using AlpacaFarm and Llama3.1-8B-Instruct.

Pos. in Inp.	#Tokens	Utility (↑)	ASR (↓)
	0	29.07	69.23
start	1	**28.44**	0.48
end	1	10.74	**0**
start	5	**28.53**	0.48
end	**5**	5.08	**0**
start	20	**29.00**	**0**
end	20	14.56	**0**

SFT, and we find that a few new embeddings are insufficient to learn that. Table 6 shows that DefensiveToken using the SecAlign loss hurts utility significantly, while achieving perfect security as in [4]. Thus, we adopt StruQ loss in our design.

Table 6: Ablation study on the loss in DefensiveToken using AlpacaFarm and Llama3.1-8B-Instruct.

Loss	Opt. Var.	WinRate (↑)	ASR (↓)
None	None	29.07	69.23
StruQ	1 token emb	**28.44**	0.48
SecAlign	1 token emb	18.70	**0**
StruQ	**5 token embs**	**28.53**	0.48
SecAlign	5 token embs	26.83	**0**
StruQ	20 token embs	**29.00**	**0**
SecAlign	20 token embs	19.61	**0**
StruQ	LoRA	**27.63**	0.48
SecAlign	LoRA	27.47	**0**
StruQ	Full	28.24	**0**

Position to insert DefensiveTokens. DefensiveTokens at the start of the LLM (before the begin_of_sentence token) is far better than those optimized and placed at the end of the input (the idea of prefilling defense [42]), see Table 7. We hypothesize that inserting them at the beginning allows them to attend to all following tokens, offering more control of the output, same as in traditional prompt tuning [14].

Learning rate turns out to affect security a lot, but not the utility, see Table 8. We tune the learning rates exponentially. 0.01 is clearly too small to lend a reasonable security. 0.1, as we used, is a good choice for security and utility. Increasing to 1 destabilize the training and may give lower or higher utility and security in an unpredictable manner.

Multiple runs show that DefensiveTokens render reliable security. We do 5 runs when optimizing different 5 DefensiveTokens for Llama3.1-8B-Instruct, and study the randomness in the 9.1K-sized SEP benchmark. We get utility WinRate `53.84 ± 0.56 (53.49, 53.75, 54.43, 53.14, 54.38)` and ASR `2.81 ± 1.09 (1.47, 4.22, 1.77, 3.79, 2.59)`.

Table 8: Ablation study on the learning rate of optimizing DefensiveTokens using AlpacaFarm and Llama3.1-8B-Instruct.

LR	#Tokens	Utility (↑)	ASR (↓)
None	0	29.07	69.23
0.01	1	**29.10**	71.63
0.1	1	28.44	**0.48**
1	1	28.18	11.06
0.01	5	**29.23**	23.56
0.1	**5**	28.53	**0.48**
1	5	27.21	3.37
0.01	20	28.72	22.60
0.1	20	28.79	7.7
1	20	**29.00**	**0**

5 Conclusion

DefensiveToken effectively mitigates prompt injection while offering the system developer the flexibility to prioritize security or utility. Compared to other test-time defenses like Reminder or Sandwich defenses, DefensiveToken reduces attack success rate by two times to an order of magnitude by asking the provider to optimize and release DefensiveTokens. Compared to other defenses that require parameter fine-tuning like StruQ and SecAlign, DefensiveToken achieves a comparable level of robustness.

DefensiveToken only defends against prompt injections, where the user (instruction) is benign, and application-retrieved external data is malicious. DefensiveToken does not apply to other safety settings, e.g., preventing jailbreaks, system following attacks, and data extraction attacks, where the user is malicious. Despite a reported trivial utility drop on AlpacaEval2 when applying DefensiveTokens, we do not know the utility on more labeled datasets, so our provided flexibility is still important, enabling developers to exclude DefensiveTokens when utility is prioritized.

Acknowledgments

This research was supported by the Google-BAIR Commons (Year 6, project 03), National Science Foundation under grant 2229876 (the ACTION center), OpenAI, Open Philanthropy, Google, the Department of Homeland Security, and IBM. We are grateful for insightful discussions and comments from Sewon Min.